

Evolução e Gestão do Ciclo de Vida de uma API

Objetivo da Aula

Conhecer os conceitos relacionados às APIs, o seu ciclo de vida e a forma de gerenciamento para uma evolução organizada.

Apresentação

Alguns conceitos se confundem na área de computação, sendo necessário entender suas diferenças para melhor utilizar as ferramentas. Esse é o caso ao compararmos bibliotecas, frameworks e APIs.

Após conhecermos as diferenças entre os três modelos de reuso abordados, iremos nos concentrar nas características das APIs, em especial na utilização de Web Services SOAP e RESTful, observando como buscam prover serviços interoperáveis, com interfaces uniformes e meios padronizados de comunicação.

Tendo em mente as características das APIs, iremos conhecer seu ciclo de vida, observando o impacto de mudanças a cada nível, com o objetivo de definir as melhores estratégias para gerenciamento e evolução das APIs, de forma a impactar minimamente os clientes da API a cada mudança ocorrida.

1. Bibliotecas, Frameworks e APIs

Os diferentes modelos de programação sempre buscaram formas de reuso para funcionalidades e serviços que se repetem ao longo do sistema. Ao definir um elemento reutilizável, além de diminuir o esforço de programação, obtemos grandes vantagens em termos de manutenção do código.

A forma mais simples de reuso é a construção de **bibliotecas**, com grupos de funções, elementos globais, classes e interfaces. A partir do momento em que esses elementos estão agrupados na biblioteca, ela pode ser importada pelos outros componentes do sistema, que tiram proveito dos recursos oferecidos.

Esse é um modelo de **reuso passivo**, em que alguns módulos de um sistema podem incorporar os recursos de outros, sem que haja qualquer interferência na sua funcionalidade. Os recursos de uma biblioteca são importados e utilizados diretamente, sem a necessidade de adotar um padrão específico no código.

Os módulos do núcleo do NodeJS são bibliotecas, importadas através de `require` ou `import`, bem como a maioria dos módulos de terceiros. Podemos citar, como outros exemplos de bibliotecas, o JQuery e o ReactJS.

Outra forma de reutilização é através de **frameworks**, que definem o modelo básico para implementação de recursos voltados para uma área específica do desenvolvimento e permitem ao programador customizar o modelo para atender às necessidades do sistema. Dois conceitos fundamentais para os frameworks são a inversão de controle e a injeção de dependências.

Como a própria expressão sugere, a **inversão de controle** ocorre quando o sistema delega para o framework a responsabilidade estrutural e funcional de um processo qualquer. Tudo que o programador tem que fazer é completar as informações necessárias e deixar que o framework cuide do processo.

Com relação à **injeção de dependência**, ela ocorre quando adicionamos um recurso gerenciado pelo framework ao código. Nesse caso, o programador que modela o processo, mas, em alguns pontos do código, o recurso incorporado é invocado, com a execução ocorrendo no contexto do framework.

Um bom exemplo de framework é o **Spring** na plataforma **Java**. A listagem apresentada a seguir demonstra sua utilização.

```
@RestController

public class PessoaController{

    @Autowired

    private PessoaRepository _pessoaRepository;

    @RequestMapping(value="/pessoa", method=RequestMethod.GET)

    public List<Pessoa> obterPessoas(){

        return _pessoaRepository.findAll();

    }
```

```
}
```

No código de exemplo, temos uma classe comum Java, com um método que retorna uma coleção de entidades a partir da invocação do método `findAll` de uma interface. As anotações `RestController` e `RequestMapping` avisam ao Spring que ele deverá criar um Web Service RESTful e responder, na rota `pessoa`, com o retorno do método no formato JSON, caracterizando a **inversão de controle**, e a anotação `Autowired` utiliza **injeção de dependência** para incluir um gestor de dados controlado pelo framework, ou seja, a consulta com `findAll` é executada no contexto do Spring.

Outro framework muito utilizado é o **Angular**, com sintaxe **Type Script**, cuja utilização é exemplificada na listagem seguinte.

```
@Component({
  selector: 'app-exemplo',
  templateUrl: './exemplo.component.html',
  styleUrls: ['./exemplo.component.css']
})
export class ExemploComponent {
  constructor(private httpClient: HttpClient) {}
}
```

Nesse segundo exemplo, a anotação `Component` determina o template que será utilizado para retornar o HTML com as informações da classe, as folhas de estilo que serão utilizadas e o seletor para uso nos demais templates, bem como o Angular, que efetua toda a ligação e o controle dos recursos, aplicando o conceito de **inversão de controle**. Já no construtor, foi definido um atributo `httpClient`, que será utilizado para receber o serviço de conexão HTTP do Angular – ou seja, aqui, temos um exemplo de **injeção de dependência**.

Finalmente, temos a **API** – ou interface de programação do aplicativo, do inglês **Application Programming Interface** –, definindo um conjunto de serviços que é oferecido por um sistema.

EXEMPLO

Um programa só pode manusear um arquivo por intermédio da API do sistema operacional, assim como podemos criar um documento de etiquetas de forma automatizada, a partir de linguagens como o C#, utilizando a API do Microsoft Word.

Quando uma API é criada, fica estabelecido um limite para os recursos que aquele sistema irá disponibilizar para o meio externo, ou seja, quem for utilizar o sistema terá acesso apenas aos serviços disponibilizados. A estrutura interna do sistema e toda a sua complexidade ficam ocultas para o utilizador.

No ambiente web, o termo API passou a caracterizar os Web Services, tanto **SOAP** quanto **RESTful**, pois tudo que precisamos saber para utilizá-los é a sua interface de serviços, embora haja um sistema muito maior por trás. Podemos citar, como exemplo, o Web Service da empresa de Correios, que fornece os dados do endereço, a partir do CEP, mas faz parte de um enorme sistema, que controla toda a logística de entrega, processo ao qual não temos acesso.

A palavra-chave em termos de APIs é **interoperabilidade**, pois seu objetivo é o de fornecer serviços para outros sistemas. Isso é garantido com a utilização de protocolos e formatos padronizados, como o XML para o SOAP e o JSON para o REST. Com a adoção de formatos conhecidos, em modo texto, e do uso do protocolo HTTP, a maioria das linguagens de programação consegue utilizar os serviços que são disponibilizados.

No ambiente do **NodeJS**, é comum a criação de APIs no estilo **REST**, tanto por meio da biblioteca padrão **http** quanto através do framework **express**.

2. Características Gerais de uma API

Ao criar suas APIs, você deverá se preocupar com alguns requisitos mínimos para que ela seja utilizada por outros desenvolvedores. Além da facilidade de uso, uma boa API deve apresentar **interface uniforme e padronizada**, bem como **versionamento** de dados e serviços.

Para que a **interoperabilidade** seja garantida, uma interface uniforme deve ser fornecida, obedecendo a padrões aceitos no mercado. Por exemplo, temos a possibilidade de usar funcionalidades de produtos Microsoft Office através de tecnologias como OLE e COM, enquanto Web Services fornecem serviços via HTTP, com dados nos formatos XML e JSON.

Uma API não representa um sistema, mas apenas um conjunto de serviços oferecidos a partir do sistema. Um dos melhores exemplos é a API do sistema operacional, já que se trata de um software responsável por gerenciar o hardware do sistema, e, apesar de efetuar operações complexas no gerenciamento dos dispositivos, oferece um conjunto limitado de serviços para os demais softwares, para que sejam capazes de utilizar esses dispositivos – ou seja, fornece uma API para que os outros sistemas sejam capazes de utilizar o hardware.

Outra preocupação é o versionamento de dados e serviços, pois o acesso a determinada API pode sofrer mudanças ao longo do tempo, e, como outros sistemas se conectam à API, eles poderiam parar de funcionar. É muito comum manter mais de uma versão da API, simultaneamente, para que os sistemas que a utilizam possam efetuar as mudanças necessárias para uma nova versão.

Por exemplo, se uma API para emissão de notas fiscais modifica o formato dos dados que serão utilizados para gerar as notas, todas as lojas que trabalham com a API deverão mudar seus sistemas. A melhor abordagem, nesse caso, é se antecipar às mudanças, disponibilizando o novo formato em um ambiente de testes, enquanto o formato antigo é mantido em produção, bem como estabelecer o prazo para que as lojas efetuem as mudanças necessárias, utilizando o ambiente de testes para ajustar seus sistemas.

Existem três tipos de APIs, em termos de utilização:

- **Privada:** é utilizada internamente na empresa, normalmente através de uma intranet, para alimentar os demais sistemas da empresa;
- **Parceiros:** viabiliza a comunicação entre duas empresas, para que possam colaborar na execução de tarefas específicas;
- **Pública:** é disponibilizada para o público em geral.

Uma API **privada** é a de gerência mais fácil, pois, além de estar indisponível na internet, facilitando o controle da segurança, costuma ser implementada pela mesma equipe de desenvolvimento que irá utilizá-la. No caso das APIs públicas e de parceiros, temos preocupações maiores com documentação, segurança e versionamento, já que ocorrerá acesso externo.

Muitas empresas dividem a construção do sistema entre o back-end, criado com tecnologias como express (NodeJS) ou Spring (Java), e o front-end, definido por meio de tecnologias como Angular ou ReactJS, ambas executando sobre o NodeJS. Ao efetuar essa divisão, os Web Services criados no back-end serão considerados uma API privada da empresa.

Para o caso das APIs de **parceiros**, será comum utilizar algum meio para a validação de acesso, já que poderão estar disponíveis na internet, e constituem a base do modelo Business to Business (**B2B**). Como serão empresas distintas e, conseqüentemente, equipes de desenvolvimento distintas, a preocupação com segurança, documentação e versionamento será uma constante.

Finalmente, as APIs **públicas** são as de gerenciamento mais complexo, pois elas podem ser acessadas de qualquer lugar, e qualquer mudança irá impactar uma grande quantidade de sistemas e equipes de desenvolvimento. Para esse tipo de API, o versionamento e a documentação são fundamentais, pois isso será tudo que as equipes de desenvolvimento dos utilizadores terão à disposição para trabalhar com a API, e a preocupação com segurança irá variar, de acordo com o tipo de serviço que é oferecido.



Destaque

Tomando como base os Web Services da empresa de Correios, que constituem uma API pública, alguns serviços não utilizam autenticação, como a busca de endereço pelo CEP, enquanto outros, normalmente voltados para empresas, utilizam um token de acesso.

No ambiente web, as APIs são construídas por meio de Web Services **SOAP** e **RESTful**. Nos dois modelos, utilizamos o protocolo HTTP e a comunicação em modo texto nos formatos JSON e XML.

Uma tecnologia emergente, que pretende substituir os Web Services do tipo RESTful, é a **GraphQL**. Ela define uma linguagem de consulta própria, que pode ser utilizada para recuperar dados específicos de uma ou mais entidades em um endpoint, em vez de estabelecer um formato de utilização fixo.

2.1. Web Services SOAP

O primeiro modelo de Web Services comercial foi o SOAP, que se baseia no formato **XML** para a garantia de **interoperabilidade**. Nesse tipo de Web Service, todos os componentes trabalham com XML, como o descritor (**WSDL**), o registro do serviço (**UDDI**) e as mensagens (**SOAP**).

Através do WSDL, é possível criar, de forma automática, o cliente para utilizar o serviço, seguindo o padrão **Proxy**, através de diversos ambientes, como a IDE NetBeans. Isso simplifica muito o processo de **integração**, além de evitar que os serviços sejam utilizados de maneira incorreta.

Embora seja o modelo mais antigo, ainda é muito utilizado em sistemas **B2B**, devido ao formalismo proporcionado pelo WSDL e ao próprio formato XML.

2.2. Web Services RESTful

Os Web Services RESTful trabalham no modelo **cliente-servidor**, por meio do protocolo HTTP, mas, ao contrário de um sistema web tradicional, utiliza todos os métodos do protocolo. Da mesma forma que qualquer requisição HTTP, não monitora estado, mas tem a possibilidade de utilizar **cache**, deixando o sistema muito mais eficiente.

Uma possibilidade das APIs REST é a utilização de **código sob demanda**, em que o servidor fornece aplicativos, como Applets Java, ou outro tipo de código executável, diminuindo a necessidade de configurações no cliente. No entanto, essa é uma funcionalidade pouco explorada, devido aos riscos que trazem para o cliente, e, na grande maioria dos casos, fornecemos apenas JSON ou XML.

A arquitetura REST trabalha em **camadas**, que é uma característica do próprio protocolo HTTP, trazendo flexibilidade e escalabilidade, além de funcionalidades como o cache compartilhado, para aumento da eficiência, e a implementação de recursos de segurança em diferentes níveis, como no proxy.

O fundamento primordial da arquitetura REST é a representação do estado da entidade por meio de **hipermídia**, preferencialmente no formato **JSON**, o que define sua **interface uniforme**. Como características do REST, temos o recurso identificado na própria solicitação, que é a rota de acesso, e o uso de mensagens autodescritivas, garantido pelo uso de pares chave-valor do JSON.

3. Evolução e Ciclo de Vida de uma API

Como no desenvolvimento de qualquer sistema, ao construir uma API, você poderá se envolver em diferentes fases do projeto. A criação da API é iniciada com o planejamento de suas funcionalidades, evoluindo a partir da ideia original, passando pelas fases de desenvolvimento e testes, até que seja implantada, utilizada e, por fim, descontinuada.

O **planejamento** de uma API é diferente de um sistema clássico, pois se deve levar em conta que seu objetivo é ser utilizada por outros sistemas. No entanto, embora o objetivo seja diferente, a metodologia ainda é a mesma em qualquer desenvolvimento de software.

Você deve manter um bom diálogo com os stakeholders da área de negócios, para que seja possível definir corretamente os requisitos de sua API, sempre mantendo uma boa documentação e passando por refinamentos sucessivos, até que um conjunto adequado de funcionalidades seja estabelecido.

Logo, devem ser levantados os requisitos do negócio, delineando os serviços que serão oferecidos pela plataforma. Também, devem ser observadas eventuais restrições técnicas que possam inviabilizar o desenvolvimento.

Ao final da fase de planejamento, o resultado deverá ser uma especificação de API, descrevendo as operações e as estruturas de dados que serão adotadas na concepção dos serviços.

Na fase de **desenvolvimento**, a especificação da API é materializada, muitas vezes, por uma equipe multidisciplinar, através de metodologias robustas, como o Scrum. Embora seja o cenário esperado para grandes empresas, não é raro que você seja o único programador e dependa da qualidade da especificação.

Os desenvolvedores precisam lidar com planejamento, codificação e testes, principalmente unitários, para que a qualidade necessária seja obtida antes da implantação da API. Em geral, são utilizados diversos produtos, como banco de dados, plataforma de programação e ferramentas de modelagem. Trabalhar com versionamento desde o início é uma boa estratégia, sendo comum a adoção do GitHub, principalmente ao lidar com vários desenvolvedores.

Alguns pontos devem ser observados ao planejar o desenvolvimento, como:

- Método de avaliação do desempenho da API;
- Mecanismo de rastreabilidade e log de erros;
- Elementos de segurança, como OAuth e uso de tokens;
- Limitações em termos de transmissão;
- Indicadores de qualidade.

Com uma versão implementada, é iniciada a fase de **testes**, na qual alguns deles já foram efetuados na etapa de desenvolvimento, como os unitários. Nessa fase, ainda temos alguns testes unitários, mas o foco principal é nos testes de desempenho e aceitação.

Os testes de desempenho devem utilizar ferramentas que permitam avaliar o uso da API em situações críticas, como grandes volumes de tráfego e variação na velocidade da conexão, por exemplo. Os resultados obtidos com os testes de desempenho permitem avaliar a eficiência de sua API em um contexto real.

Por meio de ferramentas de rastreabilidade, análise de logs e até entrevistas, os testes de aceitação permitem uma análise qualitativa, verificando se todos os requisitos foram implementados corretamente, se as estruturas de dados estão corretas e se ocorreram inclusões de requisitos. É importante uma política de testes unitários sólida, com o objetivo de minimizar as solicitações de mudanças para a equipe de desenvolvimento nessa fase.

Na fase de **implementação**, a API já deverá estar sólida e consistente, sendo disponibilizada em ambiente de produção. Para projetos mais complexos, pode ser necessário criar diferentes ambientes, nos quais o grau de maturidade é elevado, aos poucos, até que chegue ao ambiente de produção.

Uma estratégia bastante utilizada é a definição de versões, como Canary e Beta, utilizadas por usuários selecionados, funcionando como uma segunda fase de testes, antes de disponibilizar a versão final em produção. Com a utilização da API por pessoas fora da equipe de desenvolvimento, podem ser detectados eventuais erros ou necessidades tecnológicas nas plataformas dos clientes.

Talvez a fase mais complexa seja a **descontinuação** da API, mesmo que por conta de uma mudança de versão, já que diversos outros sistemas poderão estar utilizando sua API. É necessário um plano de desativação para a API, além de um monitoramento constante de seu uso, até que o impacto seja mínimo.

Em geral, as APIs evoluem continuamente, com a manutenção de eventuais erros e o acréscimo de funcionalidades. Por conta disso, é importante trabalhar com versionamento de dados e serviços, além de manter cada versão ativa, até que um plano de desativação possa ser executado, já que a oferta de múltiplas versões da API poderá gerar a necessidade de mais recursos de infraestrutura.



Destaque

Segundo o padrão para gerenciamento de versões, alterações que não quebram funcionalidades alteram apenas o valor secundário, como 1.1 e 1.2, enquanto modificações maiores alteram o número principal, como 2.0.

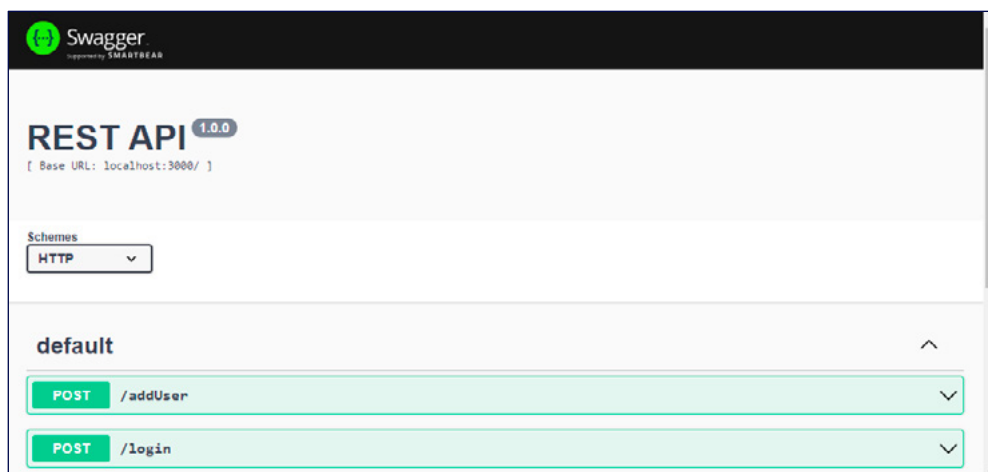
Como podemos observar, a construção de uma API exige compromisso com a continuidade, ou seja, a correção de bugs não deve alterar a interface, para que não ocorra quebra de funcionalidade, e um plano de desativação deve ser definido, para que os usuários da API possam se antecipar às mudanças.

Outro elemento importante para que a API seja adotada por outros sistemas é uma boa documentação, descrevendo os serviços e o formato de dados. Para Web Services RESTful, existe a **Open API Specification**, que documenta a API nos formatos JSON e YAML. Na listagem seguinte, você poderá observar parte de um documento no padrão Open API.

```
{
  "title": "Sample Pet Store App",
  "summary": "A pet store manager.",
  "description": "This is a sample server for a pet store.",
  "termsOfService": "https://example.com/terms/",
  "contact": {
    "name": "API Support",
    "url": "https://www.example.com/support",
    "email": "support@example.com"
  },
  "version": "1.0.1"
}
```

Uma ótima ferramenta para documentação de APIs e que adota o formato Open API é o Swagger. Seu visualizador de conteúdo, o Swagger UI, exibe o conteúdo texto da documentação na forma gráfica, além de ter um gerador, que pode ser utilizado pelo NodeJS, para criar a versão inicial do documento, a partir do código-fonte da API.

Figura 1: Visualização da documentação no Swagger



Fonte: Elaboração própria.

Considerações Finais da Aula

Após uma breve discussão acerca dos diferentes modelos de reuso, vimos o que caracteriza um componente reutilizável como API, as suas características mais comuns e como podem ser classificadas em termos de utilização. Como temos o objetivo de trabalhar em ambiente web, demos especial atenção aos modelos SOAP e REST, sempre com foco em interoperabilidade.

Analisamos o ciclo de vida da API, desde as fases iniciais, com a concepção dos serviços, até a fase do descarte, e pudemos observar as dificuldades que podem ser encontradas para evoluir uma ferramenta que é utilizada por outros desenvolvedores, em geral, de forma totalmente aberta.

Nesse contexto, vimos a importância do versionamento e da boa documentação, para que outros desenvolvedores possam utilizar nossas APIs com facilidade e para evitar a quebra de funcionalidade, nos sistemas que utilizam as nossas APIs, perante uma manutenção qualquer.

Conhecer esses detalhes fará com que sejamos mais cuidadosos nas ações relacionadas ao gerenciamento de nossas APIs, fazendo com que os serviços sejam mais consistentes e confiáveis.

Material Complementar



O que é o projeto da API REST

2023, MuleSoft. Salesforce.

O artigo aborda os elementos de uma API REST em termos estruturais, com seus fundamentos e recursos que podem ser utilizados sobre o protocolo HTTP, trazendo à tona detalhes que nem sempre são verificados.

Link para acesso: <https://www.mulesoft.com/pt/resources/api/what-is-rest-api-design> (acesso em 12 set. 2023.)

Referências

CARDOSO, L. **Frameworks back end**. São Paulo: Platos Soluções Educacionais, 2021.

Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/91769?-guid=1691770901687&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%-3fguid%3d1691770901687%26quantidadePaginas%3d1%26codigoRegistro%-3d91769%2391769&i=1>. Acesso em: 12 set. 2023.

FERREIRA, A. **Interface de programação de aplicações (API) e web services**. São Paulo: Platos Soluções Educacionais, 2021. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92072?guid=1691771178058&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691771178058%26quantidadePaginas%3d1%26codigoRegistro%3d92072%2392072&i=5>. Acesso em: 12 set. 2023.